Trainee 3: Conversational FinOps Assistant Specialist

- **Main Task:** Develop an AI-powered conversational assistant for answering FinOps queries in natural language.
- **Objective:** To provide an intuitive interface for users to get real-time cost insights without deep technical knowledge. This trainee will build the LLM-powered assistant from data connection to chat interface.
- **Estimated Total Hours:** 70 hours

Detailed Steps:

1. **Define Assistant Capabilities** (5 hours)
    - **Goal:** Clearly define what questions the assistant should be able to answer and what actions it can perform.
    - **Steps:**
        - Interview potential users (e.g., project managers, finance analysts) to understand their typical FinOps questions (e.g., "What's my project's spend this month?", "Show me historical cost for service X," "How much did we spend on networking?").
        - Prioritize common queries and categorize them by type (e.g., cost retrieval, trend analysis, anomaly reporting summary).
        - Define the scope of data the assistant will access and the expected response format (e.g., text summary, tabular data, links to dashboards).
    - **Tools:** Stakeholder interviews, Google Docs/Sheets for requirements gathering.
2. **Connect to Cost Data Sources** (15 hours)
    - **Goal:** Establish robust access to GCP billing data for the LLM.
    - **Steps:**
        - Understand the schema and content of the GCP Cloud Billing Export in BigQuery.
        - Develop optimized SQL queries to retrieve specific cost metrics and dimensions (e.g., total cost by project, daily cost by service, cost breakdown by region).
        - Create a Python-based data access layer that can execute these BigQuery queries programmatically based on LLM's "function calls".
    - **Tools:** GCP Cloud Billing, BigQuery, Google Cloud SDK, Python with `google-cloud-bigquery` library.
    - **GCP Access Needed:**
        - `bigquery.dataViewer` role (to read GCP billing export data).
        - `bigquery.jobUser` (to run queries).

3. **Train/Implement LLM Prompts or API** (15 hours)
    - **Goal:** Configure the LLM to understand FinOps queries and generate appropriate responses or actions.
    - **Steps:**
        - Select a large language model service (e.g., Google's Gemini API or Vertex AI Generative AI capabilities).
        - Design system prompts that instruct the LLM on its role as a FinOps assistant and guide its responses.
        - Implement "function calling" or "tool-use" for the LLM, allowing it to generate arguments for pre-defined functions (like the BigQuery data access layer) when a user asks a data-related question.
        - Iteratively refine prompts and function definitions to improve the LLM's understanding and response accuracy for FinOps queries.
    - **Tools:** Vertex AI Generative AI Studio, Gemini API, Python with `google-cloud-aiplatform` (Vertex AI SDK) or `google-generativeai` library.
    - **GCP Access Needed:**
        - `vertexai.user` (to interact with Vertex AI Generative AI models).
        - `serviceusage.services.use` (to use the Gemini API).
4. **Build Chat Interface (Slack, Web, etc.)** (15 hours)
    - **Goal:** Create a user-friendly interface for interacting with the FinOps assistant.
    - **Steps:**
        - Choose a target platform (e.g., a simple web application using Flask/Streamlit, or an integration with Google Chat).
        - Develop the front-end to display chat history and allow user input.
        - Implement the backend logic to receive user messages, send them to the LLM backend, and present the LLM's generated response to the user.
        - Ensure smooth message flow and error handling.
    - **Tools:** Python (Flask/Streamlit for web UI), HTML/CSS/JavaScript, Google Chat API (for Google Chat bot), Cloud Run or Cloud Functions (for hosting the backend).
    - **GCP Access Needed:**
        - `cloudfunctions.developer` or `run.developer` (for deploying the chat interface backend).
        - `chat.bot` (for Google Chat bot integration).
5. **Test with Real Queries** (10 hours)
    - **Goal:** Thoroughly test the assistant's performance with a variety of real-world FinOps questions.
    - **Steps:**

        - Compile a diverse set of test queries covering all defined capabilities.
        - Execute queries against the deployed assistant and meticulously evaluate the accuracy, relevance, and completeness of its responses.
        - Document any instances where the assistant fails, hallucinates, or provides unhelpful information.
    - **Tools:** Manual testing, Python for automated testing scripts.
6. **Deploy & Monitor Feedback** (10 hours)
    - **Goal:** Make the assistant accessible to users and gather feedback for continuous improvement.
    - **Steps:**
        - Deploy the complete conversational assistant application to a production-like environment (e.g., Cloud Run).
        - Implement logging for all user interactions and LLM responses (e.g., to Cloud Logging) to track usage and identify common query patterns.
        - Integrate a simple feedback mechanism within the chat interface (e.g., a "thumbs up/down" or a short feedback form) to collect user satisfaction and areas for improvement.
        - Set up Cloud Monitoring dashboards to observe system health and performance.
    - **Tools:** Cloud Run, Cloud Logging, BigQuery (for storing feedback), Looker Studio (for feedback dashboards).
    - **GCP Access Needed:**
        - `run.developer` (for deployment).
        - `logging.viewer` (to view logs).
        - `bigquery.dataEditor` (to store feedback).
        - `monitoring.viewer` (to view monitoring data).